

Table des matières

Arbre binaire ordonné	2
Définition	2
Propriétés	3
Recherche dans un arbre binaire de recherche	4
Version itérative de la recherche dans un arbre binaire de recherche	7
Exemple : construction d'un ABR	8
Insertion dans un arbre binaire ordonné	9
Problème.....	9
Méthode 1 : insertion en feuille.....	9
Méthode 2 itération d'insertion en feuille	11
Principe.....	11
Etapas de l'algorithme.....	11
Recherche de la Place d'insertion.....	13
Schéma récursif de la Recherche de la Place d'insertion.....	14
Algorithme réalisant l'insertion.....	17
Application : tri d'un tableau	18
Principe.....	18
Algorithme de tri	20
Recherche du plus petit(resp. Plus Grand) élément dans un ABR	22
Principe.....	22
Rechercher l'élément le plus à gauche.....	23
Recherche l'adresse du plus petit (plus grand) élément dans un ABR	25
Algorithmes de suppressions	27
Suppression du nœud racine	27
Hypothèse : non estVide(a).....	27
Principe : Suppression de la racine on distingue les cas suivants :.....	27
Exemples.....	28
Suppression d'un élément dans un arbre binaire ordonné	31
Problème.....	31
Exemples.....	32
Traduction en C.....	34
Méthode 2 : suppression d'un élément donné dans ABR donné	35
Principe.....	35
Changement de la suppression racine.....	38
Suppression de l'élément max dans un arbre binaire de recherche.....	39
Exercice	41
Intérêts des ABR	42

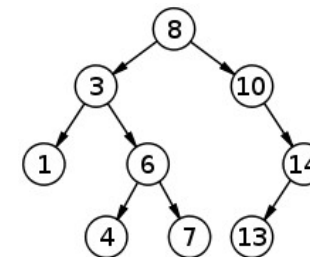
Arbre binaire ordonné

Arbre binaire ordonné = arbre binaire de recherche

Soit ($\leq$) une relation d'ordre sur l'ensemble des valeurs (données) attachées aux nœuds d'un arbre binaire.

Définition

Un arbre binaire est dit ordonné si la chaîne infixée des valeurs correspondant au parcours infixé est ordonnée.

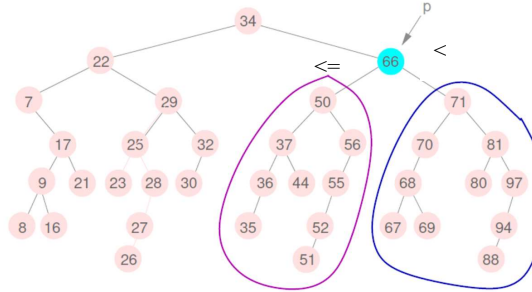
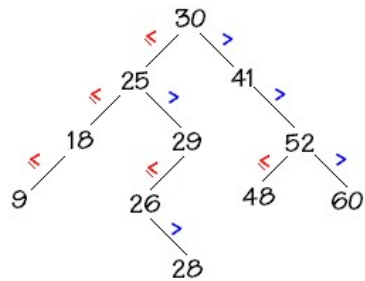
**Rappel du Parcours Infixe**

Ce parcours consiste à effectuer dans l'ordre :

- Parcours du fils gauche
- Traitement de la racine
- Parcours du fils droit

Le parcours infixé de l'arbre donne une liste ordonnée de valeurs :

[1, 3, 4, 6, 7, 8, 10, 13, 14]



Propriétés

Un arbre binaire de recherche A est un arbre qui possède les propriétés suivantes :

Si e est la donnée associée au nœud A, alors toutes les données associées au FilsGauche(A) sont inférieures ou égale à e et tous les données associées au FilsDroit(A) sont supérieures à e.

Autrement dit : chaque nœud de l'arbre A vérifié les propriétés suivantes :

Si les deux fils du nœud A sont non vides :

$$\text{donnee}(\text{filsGauche}(A)) \leq \text{donnee}(A) < \text{donnee}(\text{filsDroit}(A))$$

Si le fils gauche est vide : $\text{donnee}(A) < \text{donnee}(\text{filsDroit}(A))$

Si le fils droit est vide : $\text{donnee}(\text{filsGauche}(A)) \leq \text{donnee}(A)$

Remarque

Les deux sous arbres gauches et droit sont ordonnés.

Recherche dans un arbre binaire de recherche

Fonction $\text{rechEltABR}(\text{TElement} \times \text{Arbre}) \rightarrow \text{Booléenne}$

Fonction $\text{rechEltABR}(\text{elt}, a) \rightarrow \text{booléenne}$

Raisonnement

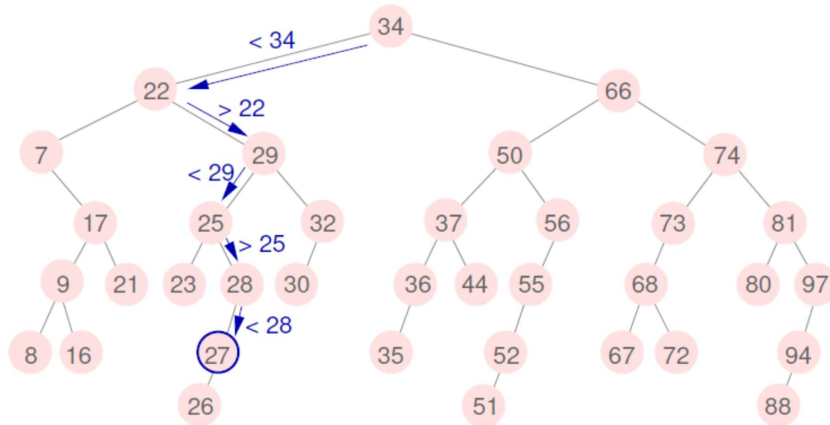
Si l'arbre

La recherche est terminée. La fonction retourne faux

Si l'arbre n'est pas vide, on distingue trois cas :

- Soit la donnée de la racine de l'arbre est égale à l'élément recherché auquel cas la recherche est terminée. La fonction retourne vraie.
- Soit la donnée de la racine est inférieure à l'élément recherché auquel cas on effectue la recherche par rapport au fils droit.
- Soit la donnée de la racine est Supérieure à l'élément recherché auquel cas on effectue la recherche par rapport au fils gauche.

Exemple



recherche de la valeur 27

Fonction rechEltABR(elt, a) → Booléenne

P.F elt : type TElement(E). a : type arbre (E).

Début

Si estVideArbre(a) alors

 Renvoyer faux

Sinon

 Si donnee(a)=elt alors

 Renvoyer Vrai

 Sinon

 Si elt < donnee(a)

 Renvoyer rechEltArbreOrdonné(elt, filsGauche(a))

 sinon

 Renvoyer rechEltArbreOrdonné(elt, filsDroit(a))

 finsi

 finsi

finsi

FIN

Version itérative de la recherche dans un arbre binaire de recherche

Fonction itRechEltABR(elt, a) → Booléenne

P.F elt : type TElement(E). a : type Arbre (E).

Début

Tantque non estVide(a) && donnee(a) ≠ elt faire

Si elt < donnee(a)

a ← filsGauche(a)

sinon

a ← filsDroit(a)

finsi

FinTq

Renvoyer non estVide(a)

FIN

Exemple : construction d'un ABR

Construction d'un arbre binaire de recherche à partir de la liste d'entiers :

[6, 8, 3, 1, 4, 9, 2, 7, 5]

On définit la construction d'un arbre binaire de recherche par l'insertion successive en respectant la propriété d'ABR.

Insertion du 6
6

Insertion du 8
6
8

Insertion du 3
3
6
8

8 > 6 → droite

3 < 6 → gauche

Insertion du 1
1
3
6
8
1 < 6 → gauche
1 < 3 → gauche

Insertion du 4
1
3
4
6
8
4 < 6 et 4 > 3

Insertion du 9
1
3
4
6
8
9
9 > 6 et 9 > 8

Insertion du 2
1
2
3
4
6
8
9
2 < 6, 2 < 3 et 2 > 1

Insertion du 7
1
2
3
4
6
7
8
9
7 > 6 et 7 < 8

Insertion du 5
1
2
3
4
5
6
7
8
9
5 < 6, 5 > 3 et 5 > 4

Insertion dans un arbre binaire ordonné

Problème

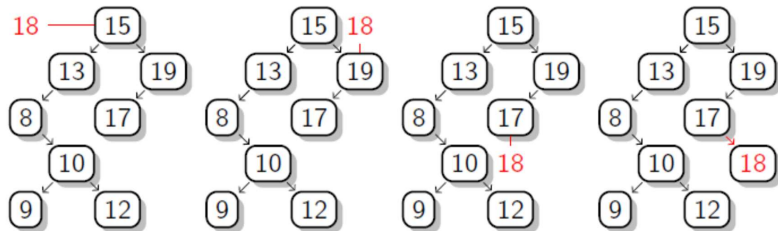
On souhaite insérer un élément donné dans un arbre ordonné tel qu'il demeure ordonné.

Fonction inserABR : TElement x Arbre → Arbre

Méthode 1 : insertion en feuille

Si l'arbre est vide, il suffit de créer une feuille contenant l'élément à insérer. Cette feuille deviendra la racine de l'arbre.

Sinon, on distingue deux cas :



- Soit la donnée de la racine est inférieure à l'élément recherché auquel cas on effectue l'insertion par rapport au fils droit.
- Soit la donnée de la racine est supérieure ou égale à l'élément recherché auquel cas on effectue l'insertion par rapport au fils gauche.

Procédure inserABR(elt , racine)

PF elt : TElement (E) et racine : Arbre (E/S)

Début

Si estVide(racine) alors

racine ← creatFeuille(elt)

Sinon

Si elt > donnee(racine) alors

inserABR(elt, filsDroit(racine))

Sinon // elt ≤ donnee(racine)

inserABR(elt, filsGauche(racine))

Finsi

Finsi

Fin

Attention : traduction en c

Méthode 2 itération d'insertion en feuille

Principe

Si l'arbre est vide, il suffit de créer une feuille contenant l'élément à insérer.

Sinon, on parcourt l'arbre, afin de rechercher un nœud qui sera le père de l'élément que nous désirons insérer.

Une fois le nœud père trouvé, il suffit de créer une feuille contenant l'élément à insérer, et l'attacher du bon côté du nœud père.

Etapas de l'algorithme

- 1) Trouver la place d'insertion
- 2) Insérer l'élément :
 - Création d'une feuille et remplissage
 - Attachement (réalisation des chaînages)

// Version itérative d'insertion d'un élément dans un arbre binaire ordonné

Procédure it_InserABR(elt, a)

P.F elt : TElement (E). a : Arbre (E/S)

Début

 Si estVide(a) alors

 a ← creatFeuille(elt)

 Sinon

 // Déterminer l'adresse d'insertion

 adrPere ← adrInserABR(elt, a)

 // On réalise l'insertion

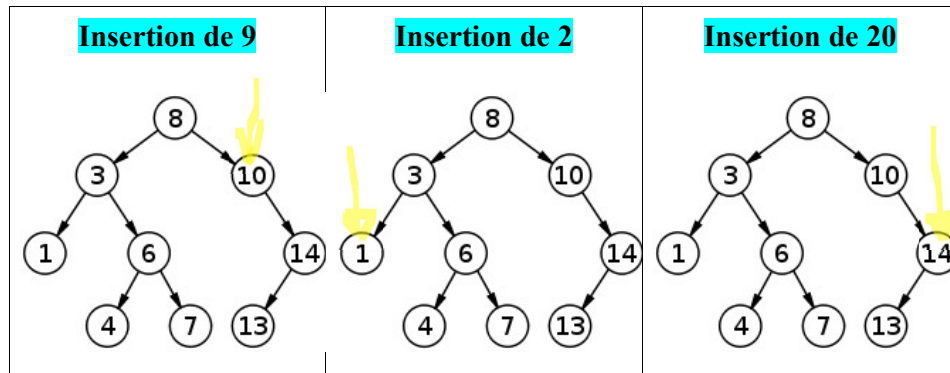
 insertion(elt, AdrPere)

 finsi

Fin

Recherche de la Place d'insertion

Exemples



On recherche un nœud de l'arbre, qui sera le père de l'élément que nous désirons insérer.

Quand nous l'aurons inséré ; ce nœud **père** est tel que :

$donnee(père) < elt$ et $estVideArbre(FilsDroit(père))$

Ou $donnee(père) \geq elt$ et $estVideArbre(FilsGauche(père))$

Ce nœud père est toujours au bout de la filiation. C'est soit une feuille soit un nœud qui n'a qu'un seul fils.

Schéma récursif de la Recherche de la Place d'insertion

On suppose que l'arbre n'est pas vide.

On distingue deux cas :

- Soit l'élément à insérer est inférieur ou égal ($\leq$) à la donnée de la racine de l'arbre $\rightarrow$ dans ce cas, l'élément fait donc partie du sous arbre gauche, on a encore deux cas :
 - Si le fils gauche est vide, on a trouvé donc le nœud qui sera le père de l'élément à insérer.
 - Si le fils gauche n'est pas vide, la place recherchée en effectuant un appel récursif sur le fils gauche.
- Soit l'élément à insérer est supérieur ($>$) à la donnée de la racine de l'arbre $\rightarrow$ dans ce cas, l'élément fait donc partie du sous arbre droit, on a encore deux cas :
 - Si le fils droit est vide, on a trouvé donc le nœud qui sera le père de l'élément à insérer.
 - Si le fils droit n'est pas vide, la place recherchée en effectuant un appel récursif sur le fils droit.

Fonction adrInserABR : Telement x Arbre → Arbre

Fonction adrInserABR-Ver1 (elt, a) → Arbre

P.F elt : type Telement (E)

a ; type Arbre (E)

Début

fg ← filsGauche(a)

fd ← filsDroit(a)

Si elt ≤ donnee(a) alors

 Si estVideArbre(fg) alors

 Renvoyer a

 Sinon

 Renvoyer adrInserABR(elt, fg)

 Finsi

Sinon // elt > donnee(a)

 Si estVideArbre(fd) alors

 Renvoyer a

 Sinon

 Renvoyer adrInserABR(elt, fd)

 Finsi

Finsi

Fin

Fonction adrInserABR-Ver2 (elt, a) → Arbre

P.F elt : type Telement (E)

a ; type Arbre (E)

Début

 Si estFeuille(a)

 Renvoyer a

 FinSi

 fg ← filsGauche(a)

 fd ← filsDroit(a)

 Si elt ≤ donnee(a) alors

 Si estVideArbre(fg) alors

 Renvoyer a

 Sinon

 Renvoyer adrInserABR(elt, fg)

 FinSi

 Sinon // elt > donnee(a)

 Si estVideArbre(fd) alors

 Renvoyer a

 Sinon

 Renvoyer adrInserABR(elt, fd)

 FinSi

 FinSi

Fin

Algorithme réalisant l'insertion

Fonction insertion TElement x Arbre → Arbre

Procédure insert(elt, adrPere)

P.F elt : TElement (E)

 adrPere : Arbre(E)

Debut

 f ← creatFeuille(elt)

 Si elt <= donnee(adrPere)

 adrPere.filsG ← f

 Sinon

 adrPere.filsD ← f

 Finsi

Fin

Application : tri d'un tableau

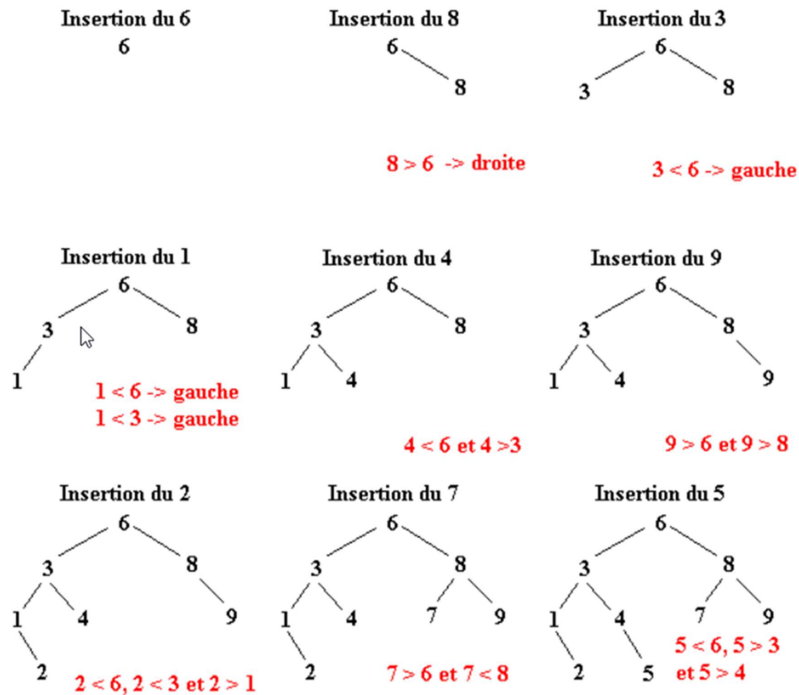
Principe

Pour trier un tableau, on procède comme suit :

- 1) Création d'un arbre binaire ordonné contenant toutes les valeurs du tableau.
- 2) Effectuer un parcours infixe de l'arbre en rangeant les éléments de l'arbre dans le tableau afin d'obtenir un tableau trié.
- 3) Libération de la mémoire occupée par l'arbre binaire ordonné

Exemple : tri du tab = [6, 8, 3, 1, 4, 9, 2, 7, 5]

- 1) Construction d'un arbre binaire de recherche à partir de la liste d'entiers :
On définit la construction d'un arbre binaire de recherche par l'insertion successive en respectant la propriété d'ABR.



- 2) Effectuer un parcours infixe de l'arbre en rangeant les éléments de l'arbre dans le tableau afin d'obtenir un tableau trié.

Tab=[1, 2, 3, 4, 5, 6, 7, 9]

- 3) Suppression de l'ABR

Algorithme de tri

Fonction triParABR : Entier x Tableau $\rightarrow$ Tableau

procédure triParABR(n, tab)

PF n : Entier (E)

tab ; Tableau (E/S)

Début

// Création de l'ABR

racine $\leftarrow$ creatFeuille(tab[1])

Pour i variant de 2 à n faire

 InserABR(tab[i], racine)

Fpour

// Parcours Infixe de l'ABR

n $\leftarrow$ 1

parcoursInfixeABRTab(n, racine, tab)

// Libération de la mémoire occupée par l'arbre binaire ordonné

suppressionArbre(racine)

Fin

Rappel Parcours Infixe

Ce parcours consiste à effectuer dans l'ordre :

- Parcours du fils gauche
- Traitement de la racine
- Parcours du fils droit

Procédure `parcoursInfixeABRTab(n, racine, tab)`

PF `n` : Entier (E/S)

`racine` : Arbre (E)

`tab` : Tableau (E/S)

Début

Si `non estVideArbre(racine)` alors

`parcoursInfixeABRTab(n, filsGauche(racine), tab)`

// Traitement de la racine

`tab[n] ← donnee(racine)`

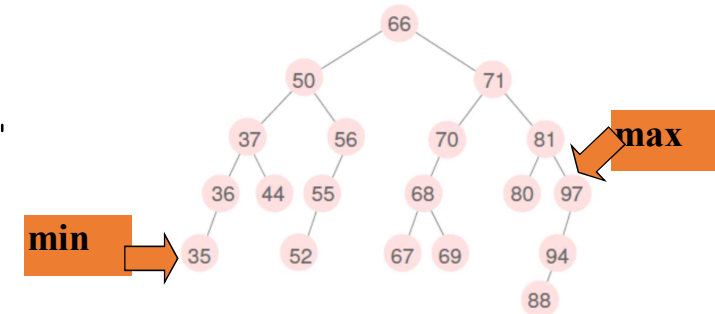
`n ← n+1`

`parcoursInfixeABRTab(n, filsDroit(racine), tab)`

finsi

Fin

Recherche du plus petit(resp. Plus Grand) élément dans un ABR



Principe

Le **plus petit élément** dans un ABR est l'élément le plus à gauche. C'est-à-dire le nœud le plus à gauche qui ne possède pas de fils gauche

Le **plus grand élément** dans un ABR est l'élément le plus à droite. C'est-à-dire le nœud le plus à droite qui ne possède pas de fils droit.

Rechercher l'élément le plus à gauche

Précondition : L'arbre est non vide

Principe :

Le **plus petit élément** dans un ABR est l'élément le plus à gauche. C'est-à-dire le nœud le plus à gauche qui ne possède pas de fils gauche

Version récursive

Fonction minABR : Arbre → TElement

Fonction minABR(racine) --> TElement

P.F racine : Arbre (E)

Début

```
fg <-- filsGauche(racine)

si estVide(fg) alors

    renvoyer donnee(racine)

sinon

    renvoyer minABR(fg)

fsi
```

FIN

Version itérative

Fonction minABR : Arbre → TElement

Fonction minABR(racine) --> TElement

P.F racine : Arbre (E)

Début

```
fg <-- filsGauche(racine)

tantque non estVide(fg) faire

    racine <- fg

    fg <-- filsGauche(racine)

finTq

renvoyer donnee(racine)
```

FIN

Recherche l'adresse du plus petit (plus grand) élément dans un ABR

Précondition : L'arbre est non vide

Principe :

Le **plus petit élément** dans un ABR est l'élément le plus à gauche. C'est-à-dire le nœud le plus à gauche qui ne possède pas de fils gauche

Version récursive

Fonction minABR : Arbre → Arbre

Fonction minABR(racine) --> Arbre

Pré- condition : L'arbre est non vide

P.F racine : Arbre (E)

Début

```
fg <-- filsGauche(racine)
```

```
si estVide(fg) alors
```

```
    renvoyer racine
```

```
sinon
```

```
    renvoyer minABR(fg)
```

```
fsi
```

Fin

Version itérative

Fonction minABR : Arbre → Arbre

Fonction minABR(racine) --> TElement

P.F racine : Arbre (E)

Début

```
fg <-- filsGauche(racine)
```

```
tantque non estVide(fg) faire
```

```
    racine ← fg
```

```
    fg <-- filsGauche(racine)
```

```
finTq
```

```
renvoyer racine
```

FIN

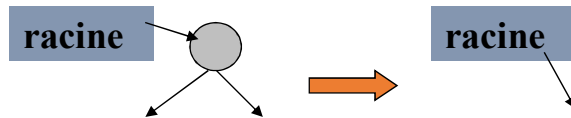
Algorithmes de suppressions

Suppression du nœud racine

Hypothèse : non estVide(a)

Principe : Suppression de la racine on distingue les cas suivants :

a) Si l'arbre est réduit à une feuille, on supprime la feuille.



b) Sinon, on remplace la racine par son fils droit s'il n'est pas vide et on rattache le fils gauche de la racine au plus petit élément de son fils droit.



Par contre on remplace la racine par son fils gauche si le fils droit de l'arbre est vide.



Remarque : Souvent la forme de l'ABR est complètement modifiée

Exemples (voir CM)

Version1 supRacine

Hypothèse : non estVide(a)

Fonction SuppRacineABR : Arbre → Arbre

Procédure SuppRacineABR(racine)

PF racine : Arbre (E/S)

Début

//Préservation du nœud à supprimer

rr ← racine

// Modification des chaînages

si estFeuille(racine) alors

racine ← initArbre()

else

fg ← filsGauche(racine)

fd ← filsDroit(racine)

Si non estVideArbre(fd) alors

//Recherche du Plus Petit Nœud de FD

ppndfd ← fd

Tantque non estVide(filsGauche(ppndfd))

faire

ppndfg ← filsGauche(ppndfd)

Fintq

ppndfd.filsg ← fg

racine ← fd

Sinon

racine ← fg

Finsi

Finsi

//libération Mémoire

LibMémoire(rr)

Fin

Version2 supRacine

Procédure SuppRacineABR(racine)

PF racine : Arbre (E/S)

Début

// Préservation du nœud racine

rr ← racine

// Modification chaînage

si estFeuille(racine) alors

racine ← initArbre()

sinon

fg ← filsGauche(racine)

fd ← filsDroit(racine)

Si non estVideArbre(fd) alors

//Recherche du Plus Petit Nœud de FD

ppndfd ← minABR(fd)

ppndfd.filsg ← fg

Racine ← fd

Sinon

racine ← fg

Finsi

Finsi

// Libération Mémoire

LibMémoire(rr)

Fin

Suppression d'un élément dans un arbre binaire ordonné

Fonction `suppEltABR : TElement x Arbre → Arbre`

Précondition : `non estVideArbre(a)`

Problème

On se propose de supprimer un élément dans un arbre binaire ordonné non vide de telle sorte que l'arbre reste ordonné.

Méthode 1 : Même principe que la recherche dichotomique

Si l'arbre est vide $\rightarrow$ c'est que l'élément à supprimer n'appartient pas à l'ABR

Arrêt de l'algorithme

Si l'arbre est non vide : on distingue les cas suivants :

- Cas où `donnee(a) = elt` $\rightarrow$ on applique la suppression du nœud racine
- Cas où `elt < donnee(a)` $\rightarrow$ on effectue la suppression par rapport au fils gauche de l'ABR.
- Cas où `elt > donnee(a)` $\rightarrow$ on effectue la suppression par rapport au fils droit de l'ABR.

Remarque : Souvent la forme de l'ABR est complètement modifiée

Exemples (voir CM)

Fonction `suppEltABR` : `TElement x Arbre` → `Arbre`

Procédure `suppEltABR(elt, racine)`

P.F

elt : `TElement (E)` **racine** : `Arbre (E/S)`

Début

Si `non estVideArbre(racine)` alors

 Si `donnee(racine)=elt` alors

`SuppRacineABR(racine)`

 Sinon

 Si `elt < donnee(racine)` alors

`suppEltABR(elt, filsGauche(racine))`

 Sinon

`suppEltABR(elt, filsDroit(racine))`

 Finsi

 Finsi

Finsi

Fin **Attention traduction en C**

Remarque : souvent la forme de l'ABR est complètement modifiée

Traduction en C (voir CM)

Méthode 2 : suppression d'un élément donné dans ABR

donné

Précondition : non estVideArbre(a)

Principe

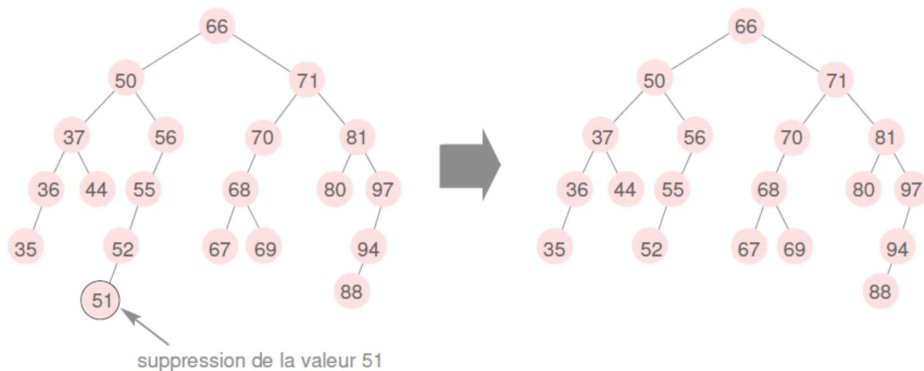
L'opération de suppression se décompose en deux étapes :

- Une étape de recherche.
- Une autre étape de suppression qui dépend de l'élément à supprimer « elt » dans l'ABR.

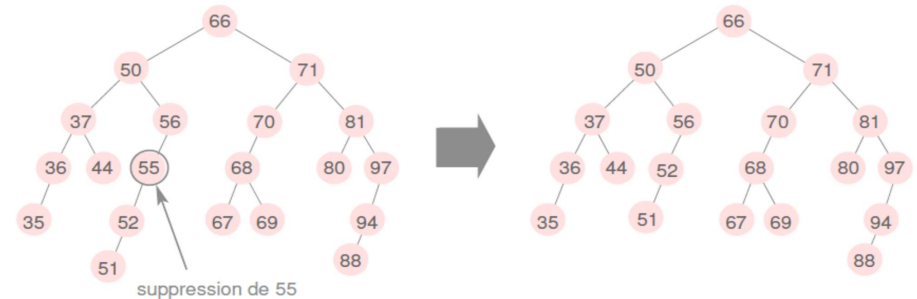
Remarque : la forme de l'ABR est légèrement modifiée

Pour cette suppression, plusieurs cas sont possibles :

- Si le nœud contenant l'élément à supprimer est une feuille, alors on supprime la feuille directement



- Si le nœud contenant l'élément à supprimer possède un seul fils, alors on remplacera le nœud par son fils, et la propriété de l'ABR ne change pas



- Si le nœud contenant l'élément à supprimer possède deux fils, plusieurs solutions sont possibles :

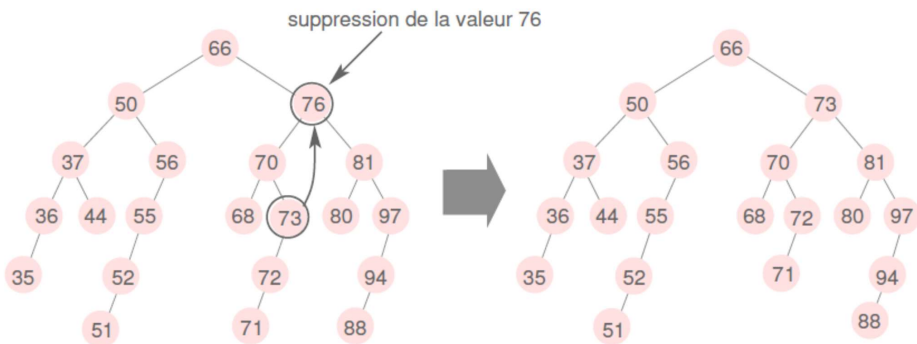
➤ On remplacera l'élément à supprimer par son **prédécesseur** ; c'est à dire le plus grand élément qui se trouve dans le sous arbre gauche du nœud contenant l'élément à supprimer.

Donnee(nœud à supprimer) $\leftarrow$ suppMax(filsGauche(nœud))

Où

➤ On remplacera l'élément à supprimer par son **successeur** ; c'est à dire le plus petit élément qui se trouve dans le sous arbre droit du nœud contenant l'élément à supprimer.

Donnee(nœud à supprimer) $\leftarrow$ suppMin(filsDroit(nœud))



Version 2 : supp Elt dans un ABR

Précondition : non estVideArbre(a)

Changement de la suppression racine

Procédure suppRacine_V2(racine)

P.F

racine : Arbre (E/S)

Début

fg $\leftarrow$ filsGauche(racine)

fd $\leftarrow$ filsDroit(racine)

Si estVide(fg) alors

rr $\leftarrow$ racine

racine $\leftarrow$ fd

libMemoire(rr)

sinon

si estVide(fd) alors

rr $\leftarrow$ racine

racine $\leftarrow$ fg

libMemoire(rr)

sinon // deux fils

eMax $\leftarrow$ suppMax(filsGauche(racine))

raccine.donne $\leftarrow$ eMax

fsi

fsi

FIN

Suppression de l'élément max dans un arbre binaire de recherche

Précondition : non estVide(racine)

Fonction supMax : Arbre → TElement x Arbre

Cet algorithme supprime le nœud contenant l'élément maximum dans un ABR (racine) donné.

Rappel : l'élément max dans un ABR est l'élément le plus à droite.

Fonction supMax(racine) → TElement

P.F racine : Arbre (E/S)

Debut

fd ← filsDroit(racine)

Si estVide(fd) alors

eMax ← donnee(racine)

//Préserver le nœud à supprimer

rr ← racine

racine ← filsGauche(racine)

libMemoire(rr)

renvoyer eMax

sinon

renvoyer supMax(fd)

fsi

FIN Attention traduction en C

Remarque : la forme de l'ABR est légèrement modifiée

Fonction supEltABR : TElement x Arbre → Arbre

Procédure supEltABR_V2(elt, racine)

P.F

elt : TElement (E)

racine : Arbre (E/S)

Début

Si non estVideArbre(racine) alors

Si donnee(racine)=elt alors

//changement proc supRacine

supRacine_V2(racine)

Sinon

Si elt < donnee(racine) alors

supEltABR_V2(elt, filsGauche(racine))

Sinon

supEltABR_V2(elt, filsDroit(racine))

Finsi

Finsi

Finsi

Fin Attention traduction en C

Exercice

Vérifier si un arbre binaire donné est un arbre binaire de recherche

Rappel : Propriété d'un ABR

Un ABR est un arbre binaire $a = [\text{racine}, \text{filsG}, \text{filsD}]$ tel que :

Les deux sous arbres filsG et filsD sont des ABRs

Et

si filsG et filsD sont non vide alors

$$\text{donnee}(\text{filsG}) \leq \text{donne}(\text{racine}) < \text{donnee}(\text{filsD})$$

si filsG est Vide et filsD non vide alors

$$\text{donne}(\text{racine}) < \text{donnee}(\text{filsD})$$

si filsD est Vide et filsG non vide alors

$$\text{donnee}(\text{filsG}) \leq \text{donne}(\text{racine})$$

Intérêts des ABR

Les Arbres Binaires de Recherche sont une structure de données qui permet de représenter correctement un ensemble ordonné de données.

Les alternatives (tableaux ou listes) ont de bien moins bons comportements dans les opérations usuelles : ajout / suppression / recherche.

Tous les algorithmes des ABRs, le temps de calcul est proportionnel à la hauteur de l'arbre (et non au nombre de données comme pour les autres alternatives)

Toutefois, dans le pire des cas, la hauteur de l'arbre est le nombre de données dans l'arbre.

En moyenne, les ABRs sont quand même souvent intéressants.

Pour éviter les mauvais cas : **équilibrage des arbres**.